Лабораторна робота 2, варіант 9: `merge`

Процесор: Intel Core i7-13700HX (8 P-ядер з HT + 8 E-ядер = 24 потоки), GCC 16.2, C++20, TBB. **Режими:** `-O0` та `-O3` при компіляції. **Дані:** рандомно згенеровані `mt19937` збережені у текстових файлах пари відсортованих послідовностей `int` у [0, 10^9], n = 10^5, 10^6, $5{\cdot}10^6$, $2{\cdot}10^7$. Час рахується як медіана 15 запусків.

Пояснення алгоритму Ми ділимо A на K майже рівних частин та записуємо їх у мектор ai(k+1). До меж (кожної) в B вираховується `bj[k] = lower_bound(B, A[ai[k]])`. Частини наших A та B зливається послідовним std::merge в K потоках. Ми їх не синхронізуєм бо результат кожної (візьмем тут k-тої) частини починається з `ai[k]+bj[k]`. області запису теж не перетинаєтья через це.

1. Бібліотечний `std::merge`, n = $2{\cdot}10^7$ + $2{\cdot}10^7$, мс

Політика	<, -O0	<, -O3	`sqrt`-предикат, -O0	`sqrt`-предикат, -O3
без політики	529.6	265.5	1028.0	661.8
`seq`	529.5	269.7	1028.3	649.1
`unseq`	529.8	271.5	1028.0	647.0
`par`	50.1	17.2	92.5	53.6
`par_unseq`	48.0	20.3	91.5	55.6

(`sqrt` - heavy comp)

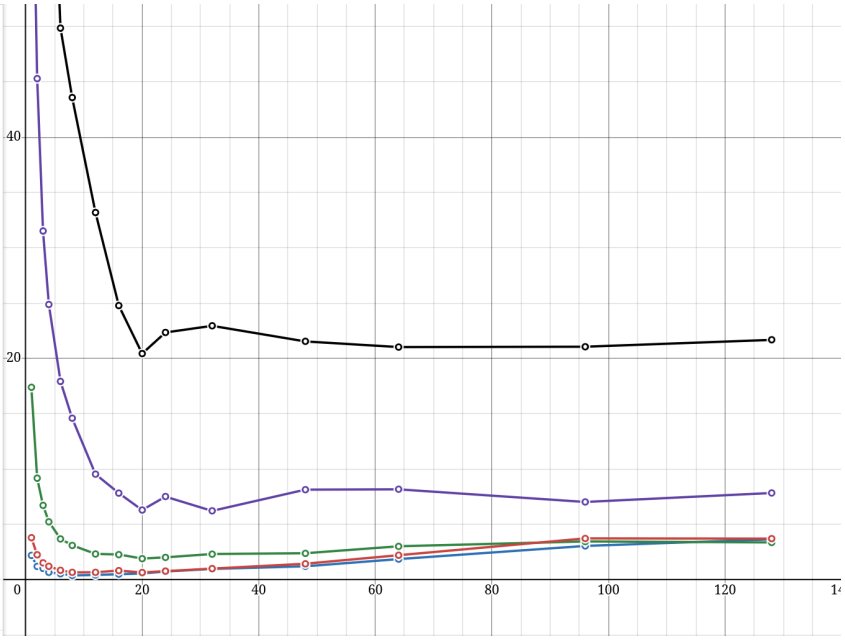
2. Власний: час залежно від K, `-O3` (останній стовбець — `-O0`)

K	n = 10^5	n = 10^6	n = $5{\cdot}10^6$	n = $2{\cdot}10^7$	$2{\cdot}10^7$, -O0
1	2.20	17.40	72.29	272.12	534.45
2	1.21	9.18	45.31	136.52	266.89
4	0.67	5.24	24.89	74.63	139.61
8	**0.39**	3.11	14.61	43.59	104.74
16	0.50	2.28	7.84	24.78	53.01
20	0.57	**1.91**	6.31	**20.45**	52.30
24	0.75	2.02	7.52	22.36	50.27
32	0.98	2.32	**6.23**	22.96	53.19
64	1.87	3.02	8.18	21.04	**49.41**
128	3.66	3.37	7.85	21.69	49.49

3. Найкраще K

n	`-O3`: K (K / 24)	`-O0`: K (K / 24)
10^5	8 (0.33)	20 (0.83)
10^6	20 (0.83)	24 (1.00)
$5{\cdot}10^6$	32 (1.33)	20 (0.83)
$2{\cdot}10^7$	20 (0.83)	64 (2.67)

x_1	y_1	z_1	u_1	v_1	w_1
1	2.2	17.4	72.29	272.12	3.8
2	1.21	9.18	45.31	136.52	2.26
3	1.03	6.72	31.52	95.17	1.53
4	0.67	5.24	24.89	74.63	1.22
6	0.55	3.69	17.93	49.87	0.85
8	0.39	3.11	14.61	43.59	0.67
12	0.44	2.33	9.55	33.2	0.68
16	0.5	2.28	7.84	24.78	0.83
20	0.57	1.91	6.31	20.45	0.66
24	0.75	2.02	7.52	22.36	0.78
32	0.98	2.32	6.23	22.96	1.01
48	1.22	2.39	8.15	21.55	1.45
64	1.87	3.02	8.18	21.04	2.22
96	3.06	3.46	7.04	21.07	3.74
128	3.66	3.37	7.85	21.69	3.72



красиві графіки

Що ми з цього вияснили: політики `seq` та `unseq` не прискорюють `merge` (як ми бачили 265.5 / 269.7 / 271.5 мс при n = $2 \cdot 10^7$), в той час як `par` і `par_unseq` прискорюють десь у 15 разів при -O3 та десь в 10 при -O0. Самі `par` і `par_unseq` між собою рівні. `unseq` доречі не прискорює бо кожен його наступний злиття залежить від результатів порівняннь попереднього. Залежність від K: до K приблизно 8 час спадає майже як T_1/K (при K = 8 ефективність десь 78%: чекали 272 / 8 = 34 мс, а виміряли 43.6 мс). Далі виграш менший і біля K ≈ 20 крива виходить на плато десь $T_1/13$ при -O3 та приблизно $T_1/10.8$ при -O0. Це мабуть через те, що `merge` майже нічого не рахує, лише читає й пише пам'ять, тому ми впираємось в цю пропускну здатність. до того ж наші 24 потоки це 8 повноцінних P-ядер з HT та 8 слабших E-ядер.

Що буде при збільшенні K далі за оптимум: додаткові потоки дають тільки накладні витрати на створення й завершення потоку, тому час росте **лінійно**: T(K) ≈ T_min + c·(K − K_opt). Для n = 10^5 (-O3): 0.98 мс (K = 32) → 1.87 (64) → 3.06 (96) → 3.66 (128), тобто c ≈ 28 мкс на потік. Для n = 10^6 нахил менший (≈ 13 мкс на потік, 2.02 → 3.37 мс при K від 24 до 128), а для n ≥ $5 \cdot 10^6$ зростання загубилось у шумі (при $2 \cdot 10^7$ усі K від 20 до 128 дають 20.5–23.0 мс): на кожен потік роботи вже багато, а накладних витрат мало.

Найкраще K і кількість потоків: у 6 із 8 випадків найкраще K лежить між 0.83 і 1.33 від кількості потоків (20–32 при 24). Для найбільшого n при -O0 плато таке пласке (K від 20 до 128 це 49.4–53.2 мс), що точний максимум (K = 64) це просто шум. Тому на практиці беремо **K ≈ кількості потоків процесора (20–24)**. Для малих n оптимум менший (8 при n = 10^5), бо накладні витрати на потоки там зрівнянні з роботою.

При n = $2 \cdot 10^7$ наш алгоритм при найкращому K (20.5 мс) лише десь на 19% повільніший за `par` (17.2 мс). Для малих n різниця більша (0.39 і 0.11 мс при n = 10^5), бо TBB перевикористовує пул потоків, а ми створюємо потоки заново при кожному виклику.

Щодо птимізації, `-O3` прискорює послідовний `merge` у 2.0 раза (530 → 266 мс), наш алгоритм при K = 1 теж у 2.0 раза, а при найкращому K у 2.4 раза (49.4 → 20.5 мс), `par` у 2.9 раза. Для `sqrt`-предиката виграш менший (1.55 раза у послідовному коді: 1028 → 662 мс), бо там більшість часу йдуть виклики `sqrt`, які компілятор оптимізує гірше.